

KitchenBuddy

Live demo script

Generative AI course · Group project · September 2026

No slides. One laptop on the projector, four voices.

How to use this script

We present by doing. The MacBook Air runs the app on the projector, and the story follows a normal evening: a receipt, a tired request for dinner, and cooking with the chef. The technical explanations fit into the moments where the laptop is working anyway, so the waits become the talk.

Who does what. The DevOps Engineer drives the laptop for the whole demo, speaks the voice commands, and opens and closes. The others speak at their parts. Nobody reads: the "Say" text is what we want to get across, in words that sound natural. Say it in your own words once you know it.

Timing. The full run takes 7 to 8 minutes, depending on how fast the laptop is that day. If we only get 5, follow the cuts in part 3.

#	Part	Speaker	Time	On screen
1	A fridge at 7 pm	DevOps	0:00–0:40	Today screen
2	What runs on this laptop	DevOps	0:40–1:20	Kitchen screen
3	Scanning a receipt	Data Engineer	1:20–2:50	Camera, then the read receipt
4	"Bob, I'm drained"	AI Researcher	2:50–3:50	Recipe being written
5	Why the recipe can be trusted	ML Engineer	3:50–4:40	The recipe
6	Cooking with the chef	DevOps + ML Engineer	4:40–5:50	Cooking mode
7	Fine-tuning, and what doesn't work yet	AI Researcher	5:50–6:45	Cooking mode stays open
8	Close	DevOps	6:45–7:10	Today screen

Run of show.

Part 1. Before we go on

The day before

- **Do two full rehearsals on the demo Mac, with the same stock we will use on the day.** The model does not answer exactly the same way twice, so learn what it usually says.
- **Fill the kitchen for a good quick dinner.** For example rice, eggs, tomatoes, an onion, a courgette, chicken, cheese, olive oil. Avoid a near-empty kitchen: the recipe will be weak.
- **Get a real supermarket receipt,** flat, not folded, printed clearly. Also keep a sharp JPEG photo of it in Pictures, in the Receipts folder, as a backup.
- **Decide on answer prefetching** during rehearsal. If the chef is slow to answer in cooking mode, set `PREFETCH_ENABLED=false` in the backend configuration.

Thirty minutes before

1. Plug in the charger. Close every app we don't need, because the model and the voices share the 16 GB.
2. Open LM Studio. Load `gemma-4-e2b-it-mlx`, start the server, and check that the context length is 8192 and thinking is off.

3. Start the backend without auto-reload, so nothing restarts mid-demo: in happy-bite/backend, run `./run.sh --no-reload`. Wait for the log line "voice: warm in ...s".
4. Open the app in Chrome at `http://localhost:5173` (or `http://127.0.0.1:8000` if the frontend is built). Use "localhost", not the network IP address: the camera only works on localhost or https.
5. Connect the laptop's sound to the room speakers so the jury hears the chef. Test the microphone by saying "Bob" once.
6. Check the Wi-Fi. The wake word "Bob" needs internet, because Chrome's recogniser is online.
7. Turn off Center Stage from the Video Effects menu in the macOS menu bar, otherwise the camera zooms in on faces.
8. Set Chrome to full screen and zoom to 125 % so the back row can read.
9. Go to the Today screen and leave it there.

Things we don't do on stage

- **Don't chain follow-up questions in cooking mode.** After a few exchanges the chef can drift to the wrong step (we measured it). Ask one question about the current step, then say "Bob, next".
- **Don't use "Any ingredients".** It writes dishes that need shopping, and takes longer.
- **Don't hold the receipt too close** to the camera. It has fixed focus and the photo goes blurry. Check the review screen before pressing "Use this photo".
- **Don't import an iPhone HEIC photo.** Only JPEG, PNG and WebP are read.

Part 2. The demo

1. A fridge at 7 pm 0:00–0:40 · DevOps

Do

Stand next to the laptop. The Today screen is on the projector. Don't touch anything yet.

Say

Good afternoon. Picture a normal evening. It's seven o'clock, you open the fridge, and there's half a courgette, some rice, two chicken thighs that really need to be eaten tomorrow, and you are exhausted.

Recipe apps give you a book to search. Shopping apps give you a list. Nobody gives you someone standing next to you who knows what's on your shelves and keeps talking while your hands are busy. That's what we built. It's called KitchenBuddy, and everything you're about to see runs on this laptop.

2. What runs on this laptop 0:40–1:20 · DevOps

Do

Tap the Kitchen tab. Scroll slowly through the stock so the expiry colours are visible.

Say

This is our kitchen: what we have and how long each thing keeps. This part uses no AI at all. It runs in the browser and still works with the server switched off.

The AI part runs on this MacBook Air: an M4 chip and 16 gigabytes of memory, shared by everything. That's the constraint the course gave us, running the model ourselves instead of calling an API, and it shaped every decision you'll hear about. On this machine: a language model, Gemma 4 E2B from Google; a speech recogniser; and a voice. No cloud account, no data leaving the room.

3. Scanning a receipt 1:20–2:50 · Data Engineer speaks, DevOps drives

Do

Tap the + button, then "Scan a receipt". The camera opens.

Hold the receipt flat, about 30 cm from the camera, filling the dashed frame. Press the round shutter button.

Look at the review screen. If the prices are readable, press "Use this photo". If not, press "Retake".

Say

We came home from shopping, so let's add what we bought. Reading a receipt takes up to a minute, so while it works, here's what's happening.

First, Tesseract, an open-source OCR engine, turns the photo into lines of text. But a line like "CRF LT DEMI ECR 1 L" means nothing to a small model. So we built a product database from Open Food Facts: 483,000 products sold in France, and 330,000 of them linked to one of the 3,000 ingredients our app knows. Each line is searched by pieces of words, so "ECR" can find "écrémé".

Then the language model reads each line together with what the database found, and decides. When the model and the database disagree, the app doesn't guess: it marks the line "Check" and asks you. A small anecdote: the first time we tried it, Gemma put 2.31 courgettes in the fridge. That was the price.

Do

When the receipt appears, point at one matched line and one "Check" line if there is one. Tap "Put ... items away".

If it goes wrong

If the camera doesn't start or the photo is blurry, say "let's use the photo we took this morning", press "Import a receipt" and pick the JPEG from Pictures, then Receipts. If reading fails completely, press "Use a sample receipt" and keep talking. The explanation stays the same.

4. "Bob, I'm drained" 2:50–3:50 · AI Researcher speaks, DevOps says the command

Do

Go back to Today. DevOps says clearly: "Bob, I'm drained. Give me fifteen minutes." The app starts writing a recipe, which takes 20 to 30 seconds.

Say

While it writes, a word about the model. We didn't pick it from a leaderboard. We started with Qwen 2.5 7B on paper, but on 16 gigabytes it left no room for the voice. We tried Qwen 3.5 9B, which is a reasoning model, and it thought for 26 minutes before writing one recipe. The small models were fast but wrote bad recipes: one asked for 380 grams of salt for three people.

Gemma 4 E2B was the balance: about three and a half gigabytes in 4-bit, around 15 words per second on this laptop, and good enough recipes. Its trick is that it has 5 billion parameters but runs like a 2 billion one, because much of it sits in cheap lookup tables. We also run it colder than Google recommends, temperature 0.2 for recipes, because we want reliable output, not creativity with the commas.

If it goes wrong

If "Bob" isn't heard, say it once more, closer to the laptop. If it still doesn't work, the Wi-Fi is probably down: open the chef chat from the + button and type the same sentence. Say "the wake word needs internet, that's one of our limits" and move on.

5. Why the recipe can be trusted 3:50–4:40 · ML Engineer

Do

The recipe opens. Scroll slowly: ingredients, then steps with heat and cues.

Say

Everything in this recipe was checked before you saw it. A small model makes mistakes, so we treat its answer like input from a stranger. The server forces the answer to follow a fixed JSON format. Then plain Python code checks it: do the times add up to fifteen minutes? Does a step mention a food that isn't in the recipe? Are the quantities something a person can eat? If something is wrong, the model gets the exact list of problems and one more try.

The recipes also don't come out of nowhere. We indexed 2.1 million real recipes. We first tried embeddings, the classic approach from the course, but indexing 2 million recipes on a laptop takes hours. So we used full-text search in SQLite. Our first version took 3.3 seconds per search. Keeping only the five rarest words in the question brought it down to a few milliseconds.

6. Cooking with the chef 4:40–5:50 · DevOps drives, ML Engineer speaks

Do

Press "Start cooking". The chef reads step 1 aloud. Let it finish.

DevOps says: "Bob, next." The chef reads step 2.

DevOps asks one question about this step, for example: "Bob, how do I know it's ready?"

Say

This is the part we use most: hands-free cooking. Everything you hear is generated here. The speech recogniser runs on the laptop's graphics chip, and the voice is Kokoro, an 82-million-parameter model. We picked it over a lighter one, Piper, because something that talks to you for twenty minutes shouldn't sound like a robot.

The answer is spoken sentence by sentence while it's still being written, so you don't wait for the whole reply before hearing it. And while you chop, the server quietly prepares answers to the questions each step usually raises, so some answers are instant.

If it goes wrong

If the chef answers about the wrong step, don't fight it. Say "Bob, repeat" and use it as the bridge into part 7: it's exactly the bug we measured.

7. Fine-tuning, and what doesn't work yet 5:50–6:45 · AI Researcher

Do

Leave cooking mode open on screen.

Say

You might ask why we didn't fine-tune. We did the maths from the course. The knowledge here is a fridge that changes every day and 2 million recipes. That belongs in retrieval, not in weights. For the behaviour, LoRA would train about 40 million parameters, half a percent of a 7B model, in an hour or two on one gaming GPU. A full fine-tune would need about 120 gigabytes of GPU memory. We wrote and tested the whole pipeline, but our only GPU was this laptop, and the checks had already removed most of the mistakes a fine-tune would fix. So it's our next step, not something we'll pretend we did.

And to be honest about the rest: after a few questions, the chef can drift to the wrong step. We measured it. With the conversation history it answered about the wrong step in all 6 runs we tried; with the current step repeated in the question it got it right in all 6. We know the fix, and it isn't shipped yet. The wake word also needs internet, and a laptop camera is not a great scanner.

8. Close 6:45–7:10 · DevOps

Do

Say "Bob, stop cooking". Go back to the Today screen.

Say

So that's KitchenBuddy: a receipt, a tired request, and a chef that talks you through dinner, all on a laptop, with nothing leaving the room. What we learned most is that with a small model, choosing it is the easy part. Making it reliable is the real work. Thank you, we're happy to take questions, and to try something live if you want.

Part 3. If we only have 5 minutes

Part	What to cut	New time
1. A fridge at 7 pm	Keep as is.	0:35
2. What runs on this laptop	Skip the Kitchen screen. Say one sentence about the MacBook Air and the three models while standing on Today.	0:15
3. Scanning a receipt	Keep the OCR and database explanation. Drop the courgette anecdote.	1:10
4. "Bob, I'm drained"	Keep the 26-minute story and the Gemma numbers. Drop Qwen 2.5 and the temperature.	0:50
5. Why the recipe can be trusted	Keep the checks. Retrieval in one sentence: "2.1 million real recipes, full-text search, milliseconds".	0:40
6. Cooking with the chef	"Start cooking" and "Bob, next" only. No question.	0:40
7. Fine-tuning and limits	LoRA numbers in two sentences. One limit: the step drift.	0:35
8. Close	Keep as is.	0:15

Five-minute version, total about 5:00.

Part 4. When something breaks

A live demo will surprise us at least once. The jury cares more about how we handle it than about a perfect run. Stay calm, say what happened in one sentence, use the fallback, keep going.

What happens	What we say	What we do
The chef takes more than a minute	"It's a laptop, the model is reading its prompt. That's exactly why we cap the context at 8,000 tokens."	Wait up to 90 s. If nothing comes, go to the Recipes tab and open a recipe we saved during rehearsal.
"Bob" isn't heard	"The wake word runs on Chrome's online recogniser."	Type the request in the chef chat instead.
The camera shows nothing	"Let's use the photo we took this morning."	Import a receipt, then Pictures, then Receipts.
The receipt reads badly	"Laptop cameras aren't scanners, that's on our limits list."	Use a sample receipt.
A strange recipe	"This is why every recipe goes through checks. Let's see what they caught."	Point at the "check this before you start" box if it appears. Otherwise open a saved recipe.

What happens	What we say	What we do
No sound from the chef	"The voice is on, the room isn't."	Check the output device; the text is on screen anyway.
The backend crashes	"The core of the app doesn't need the server, look."	Show the Kitchen and Shopping screens, which still work. Restart with <code>./run.sh --no-reload</code> .

Fallbacks.

Part 5. Questions the jury may ask

Question	Short answer
Why not just use ChatGPT or Claude?	The course asked us to run the model ourselves. It also makes the product better in some ways: your fridge stays private, it costs nothing per question, and it works offline, apart from the wake word.
Why RAG and not fine-tuning?	The knowledge changes every day or needs a source: a fridge and 2 million recipes. The course says that's retrieval. Fine-tuning fixes behaviour, and we computed it: 40 million parameters, a couple of GPU hours, stage two.
Why Gemma 4 E2B?	It was the best balance on 16 GB: about 3.6 GB, 15 tokens per second, good enough recipes, room for voice. Bigger models starved the voice or thought for 26 minutes; smaller ones wrote bad recipes.
What does "2.3 billion effective, 5.1 billion total" mean?	Much of the model is in per-layer embedding tables that are cheap to hold and compute. It has the knowledge of a 5B model at roughly the cost of a 2B one.
The card says 128K context. Why only 8,000?	Because on a laptop every prompt token is read before the answer starts, and the cache shares memory with the voice. The window that matters is the one the user is willing to wait for.
Why not embeddings for retrieval?	We built that prototype. Embedding 2.1 million recipes takes hours on a laptop. Our queries are mostly ingredient lists, which word search handles well. Re-ranking the top results with embeddings is our next step.
How do you stop hallucinations?	Real recipes as reference, output forced into a schema, and Python checks on times, ingredients and portions, with one retry. What's still wrong is shown to the user, not hidden.
How did you evaluate it?	Not with perplexity: there's no reference answer for "dinner from this fridge". We measured speed, search time and recipe checks, and ran small controlled tests, like the step-drift experiment.
Does 4-bit hurt quality?	A little. It's what lets a 5-billion-parameter model fit in 3.6 GB. We judged it on our own task, and the recipes were good enough.
What about French?	Gemma speaks French, the speech recogniser can be switched to French, and Kokoro has one French voice. The recipe corpus is in English, so the model translates as it writes.
What would you do with another week?	Ship the step-drift fix, run the LoRA on an NVIDIA machine and compare with the untuned model, add embedding re-ranking, and move the wake word onto the device.

Part 6. Numbers to know by heart

Figure	Value
Demo machine	MacBook Air, Apple M4, 16 GB unified memory

Figure	Value
Model	Gemma 4 E2B, 4-bit: 2.3 B effective / 5.1 B total parameters, 3.55 GB, about 15 tokens/s
Context	128K on the card, prompts capped at 8,192 tokens, 35 % kept for the answer
Worst model incident	Qwen 3.5 9B reasoned for 26 minutes before one recipe
Recipes indexed	2,119,522, full-text search in SQLite, 3.3 s down to milliseconds
Receipt database	483,210 French products, 329,783 linked to our 3,052 ingredients
Quick dinner	20–25 s to write
LoRA	rank 16, about 40 million trainable parameters, 0.53 % of a 7B model; full fine-tune about 120 GB
Voice	Parakeet 0.6B hears (on the Mac GPU), Kokoro 82M speaks, a short phrase takes 0.15–0.25 s to synthesise
Step-drift experiment	With history: 0 of 6 runs on the right step. Step repeated in the question: 6 of 6.